

AI-Assisted Box Selection System Plan

<https://gemini.google.com/app/375d48c2be22597c>

User prompt: 📦 *What the Project Is About* You are building an AI-Assisted Box Selection System for an e-commerce warehouse. When customers buy items, the warehouse team needs to package them. To keep shipping efficient and safe, the system needs to evaluate available shipping boxes and automatically recommend the best box based on constraints. ⚠️ *The Core Constraints* **Dimensions & Volume:** Each product and box has length, width, and height. Products must fit inside the box's internal dimensions. **Weight Limit:** Products have weight, and each box has a max_weight capacity that cannot be exceeded. **Cost Optimization:** Different boxes have different costs. Out of all the boxes that fit the items and safely support their weight, the system must choose the cheapest box. 🧠 *What Their Recommendations & Expectations Are* The company is evaluating both your Django/algorithm coding skills AND your ability to collaborate with AI responsibly. Here are their key expectations: 1. You ARE Allowed (and Encouraged) to Use AI You can use tools like ChatGPT, Claude, Cursor, or GitHub Copilot to help write code, generate test cases, or design models. 2. Proof of Critical Thinking (The AI Audit) They want to verify that you aren't blindly copy-pasting AI output. You must document your process in an AI_USAGE.md file, covering: Which AI tools you used. The prompts you used. Code output you accepted vs. rejected/modified. Mistakes or bugs the AI made. How you tested and verified that the code actually worked. 3. Absolute "No AI Generation" Guardrail Chat Transcripts: You must attach full raw exports of your chats with AI in your repo. "What Did You Learn?" Section: This reflection must be written 100% by you. If they detect AI-generated text in your personal reflection, the assignment will be rejected immediately. 4. Required Deliverables GitHub Repository Link README.md (Project setup, description, and "What I Learned" reflection) AI_USAGE.md (Your detailed AI interaction tracking) Chat Transcripts (Exported chat files in the repository) Unit Test Cases TEST_OUTPUT.md or a GitHub Actions CI link showing tests passing , lets plan first

Response: Here is a complete, step-by-step roadmap to build and submit your **AI-Assisted Box Selection System** while fully meeting all hiring requirements.

🔧 Step 1: Core Algorithm & System Architecture

Before touching Django, define how box selection works to keep your logic clean and testable.

1. Data Models

- **Product:** id, name, length, width, height, weight
- **Box:** id, name, length, width, height, max_weight, cost
- **Order:** id, created_at, list of Product items

2. Selection Logic Strategy

To find the **cheapest valid box**:

1. **Weight Check:** Sum total weight of items in order $\leq$ Box max_weight.
2. **Volume Check:** Sum volume of items $\leq$ Box volume (Length $\times$ Width $\times$ Height). (*Baseline filter*)
3. **3D Packing/Fitting Check:** Verify item dimensions actually fit inside the box's internal boundaries (accounting for orientation/rotation).
4. **Optimization:** Filter down to all candidate boxes that pass step 1–3, then select the one with the lowest cost.

📅 Step 2: Implementation Plan

Phase	Milestone	Deliverables / Action Items
Phase 1	Django Setup & Models	Initialize app, define Product, Box, Order models with basic validation.
Phase 2	Recommendation Engine	Implement box selection logic (3D fitting or simplified bounding-box heuristic + weight/cost check).

Phase	Milestone	Deliverables / Action Items
Phase 3	API / Web Views	Build Django REST API or Django View endpoint (/api/recommend-box/) accepting order items and returning the best box.
Phase 4	Testing Suite	Write Django unit tests covering single item, multi-item orders, overweight edge cases, and tie-breaking on cost.
Phase 5	CI & Output Docs	Set up GitHub Actions workflow (or run tests locally and export output to TEST_OUTPUT.md).

Step 3: Required Repository Structure

```
box-selection-system/
├── box_app/                # Django App
│   ├── models.py
│   ├── views.py
│   ├── services.py        # Box selection algorithm logic
│   └── tests.py           # Comprehensive Unit Tests
├── docs/
├── [cite_start] chat_transcripts/ # Exported raw chat files (.json / .markdown) [cite: 21]
├── .github/
├── [cite_start] workflows/ci.yml  # GitHub Actions (Optional, but recommended) [cite: 26]
├── [cite_start] README.md         # Setup, architecture & personal reflection [cite: 19, 23]
├── [cite_start] AI_USAGE.md       # Transparent tracking of AI usage [cite: 9, 20]
└── [cite_start] TEST_OUTPUT.md    # Formatted test execution output [cite: 25, 27]
```

Step 4: AI Usage Strategy (Keeping It Transparent)

To fulfill the strict **AI Audit** requirements without risking rejection:

- 1. Keep Prompts Granular:** Prompt AI for modular components (e.g., *"Write a Python helper function to check 3D bounding box constraints"* or *"Generate pytest fixtures for Django Box models"*).
- 2. Log Interactions in Real Time:** Keep a running log or save chat logs directly as you work.
- 3. Document Modifications:** Explicitly record in `AI_USAGE.md` where the AI missed edge cases (e.g., AI forgot to account for 3D rotation, or generated inefficient database queries).

Step 5: Draft Outlines for Mandatory Files

`AI_USAGE.md` Draft Outline

- **Tools Used:** (e.g., Cursor, Claude 3.5 Sonnet, ChatGPT)
- **Key Prompts Used:** Document exact prompt text for key modules.
- **Accepted vs. Rejected Outputs:** List what code was kept vs. rewritten manually.
- **AI Errors Found:** Details on hallucinations, incorrect dimension checks, or missing edge cases.
- **Verification Methods:** Running `python manage.py test` and manually asserting math edge cases.

`README.md` Draft Outline

- **Project Title & Overview**
- **Local Setup & Installation Instructions**
- **API Endpoints & Algorithm Strategy**
- **Personal Reflection ("What Did You Learn?"):**

⚠ Write this section 100% manually. Focus on practical lessons—like trade-offs in 3D bin packing algorithms, handling constraints in Django, or lessons learned when auditing AI-generated code.

User prompt: *"I need to set up a new Django REST API project called box_selection_system using SQLite3. Can you give me the step-by-step terminal commands to: Create a virtual environment (venv) and activate it. Install Django and Django REST Framework (djangorestframework). Create the main Django project structure inside my root directory. Create a Django app named box_app and register it in settings.py. Generate a requirements.txt file."*

Response: Here are the step-by-step commands to set up your project structure.

Step 1: Create and Activate Virtual Environment

Open your terminal in your root project folder and run:

```
# Create virtual environment
python3 -m venv venv

# Activate virtual environment
# On macOS / Linux:
source venv/bin/activate

# On Windows (Command Prompt):
venv\Scripts\activate.bat

# On Windows (PowerShell):
venv\Scripts\Activate.ps1
```

Step 2: Install Django and Django REST Framework

With the virtual environment activated, install Django and djangorestframework:

```
pip install django djangorestframework
```

Step 3: Create the Main Django Project Structure

Create the project in your current directory using `.` so you don't end up with nested directories:

```
django-admin startproject box_selection_system .
```

Step 4: Create the box_app Django App

Create the application folder:

```
python manage.py startapp box_app
```

Now, register both `rest_framework` and `box_app` in `box_selection_system/settings.py` under `INSTALLED_APPS`:

```
INSTALLED_APPS = [
    # Default Django apps...
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Third-party packages
    'rest_framework',

    # Local apps
    'box_app',
]
```

(Note: SQLite3 is configured by default in `settings.py` under `DATABASES`, so no additional database configuration is required).

Step 5: Generate requirements.txt

Freeze your installed dependencies into `requirements.txt`:

```
pip freeze > requirements.txt
```

Step 6: Initial Migration & Server Verification

Run initial migrations to create standard SQLite tables and launch the server:

```
python manage.py migrate
python manage.py runserver
```

User prompt: "Generate the `box_app/models.py` file with models for `Product`, `Box`, `Order`, and `OrderItem`. Use `DecimalField` for precise dimensions (`length`, `width`, `height`), `weight`, and `cost`. Include `MinValueValidator` to prevent zero or negative values. Add `@property` helper methods on `Product` and `Box` to calculate volume. Add helper methods on `Order` to calculate the total cumulative weight and total volume across all `OrderItem` items."

Response: Here is the implementation for `box_app/models.py` with field validation, volume properties, and order-level aggregation methods.

`box_app/models.py`

```
from decimal import Decimal
from django.db import models
from django.core.validators import MinValueValidator

# Common validator to ensure physical measurements are strictly positive (> 0)
POSITIVE_MIN_VALUE_VALIDATOR = [MinValueValidator(Decimal('0.01'))]

class Product(models.Model):
    """
    Represents an individual item that can be ordered.
    Dimensions are in centimeters (cm), weight in kilograms (kg).
    """
    name = models.CharField(max_length=255)
    length = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        validators=POSITIVE_MIN_VALUE_VALIDATOR,
        help_text="Length in cm"
    )
    width = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        validators=POSITIVE_MIN_VALUE_VALIDATOR,
        help_text="Width in cm"
    )
    height = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        validators=POSITIVE_MIN_VALUE_VALIDATOR,
        help_text="Height in cm"
    )
    weight = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        validators=POSITIVE_MIN_VALUE_VALIDATOR,
        help_text="Weight in kg"
    )
```

```

@property
def volume(self) -> Decimal:
    """Calculates volume in cubic centimeters (cm³)."""
    return self.length * self.width * self.height

def __str__(self):
    return f"{self.name} ({self.length}x{self.width}x{self.height} cm)"

class Box(models.Model):
    """
    Represents a shipping container available in the warehouse.
    Internal dimensions in cm, max_weight capacity in kg, cost in currency units.
    """
    name = models.CharField(max_length=255)
    length = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        validators=POSITIVE_MIN_VALUE_VALIDATOR,
        help_text="Internal length in cm"
    )
    width = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        validators=POSITIVE_MIN_VALUE_VALIDATOR,
        help_text="Internal width in cm"
    )
    height = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        validators=POSITIVE_MIN_VALUE_VALIDATOR,
        help_text="Internal height in cm"
    )
    max_weight = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        validators=POSITIVE_MIN_VALUE_VALIDATOR,
        help_text="Maximum weight capacity in kg"
    )
    cost = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        validators=[MinValueValidator(Decimal('0.00'))],
        help_text="Cost of the box"
    )

    @property
    def volume(self) -> Decimal:
        """Calculates internal volume in cubic centimeters (cm³)."""
        return self.length * self.width * self.height

    def __str__(self):
        return f"{self.name} - ${self.cost} (Max Wt: {self.max_weight}kg)"

class Order(models.Model):
    """
    Represents a customer order containing one or more products via OrderItem.
    """
    created_at = models.DateTimeField(auto_now_add=True)

    @property
    def total_weight(self) -> Decimal:
        """Calculates total cumulative weight across all order items."""

```

```

        items = self.items.select_related('product')
        return sum((item.product.weight * item.quantity for item in items), Decimal('0.00'))

    @property
    def total_volume(self) -> Decimal:
        """Calculates total cumulative volume across all order items."""
        items = self.items.select_related('product')
        return sum((item.product.volume * item.quantity for item in items), Decimal('0.00'))

    def __str__(self):
        return f"Order #{self.id} ({self.created_at.strftime('%Y-%m-%d %H:%M')})"

class OrderItem(models.Model):
    """
    Junction table representing products within a specific order and their quantities.
    """
    order = models.ForeignKey(
        Order,
        related_name='items',
        on_delete=models.CASCADE
    )
    product = models.ForeignKey(
        Product,
        on_delete=models.PROTECT
    )
    quantity = models.PositiveIntegerField(
        default=1,
        validators=[MinValueValidator(1)]
    )

    def __str__(self):
        return f"{self.quantity}x {self.product.name} (Order #{self.order.id})"

```

Key Highlights

- 1. Validation:** `MinValueValidator(Decimal('0.01'))` is attached to all dimensional, weight, and quantity fields to prevent zero or negative values at the DB/Form level.
- 2. Precision:** All monetary and measurement values utilize `DecimalField` to eliminate floating-point rounding errors.
- 3. Performance Optimization:** `total_weight` and `total_volume` helper methods utilize `.select_related('product')` to avoid N+1 database queries when iterating over `OrderItem` instances.

User prompt: "Write a python service module in `box_app/services.py` for a box recommendation system. Requirements: Create a function `recommend_box_for_order(order: Order) -> Optional[Box]`. Filter out boxes where `order.total_weight()` exceeds `box.max_weight`. Filter out boxes where `order.total_volume()` exceeds `box.volume`. Ensure individual item dimensions fit inside the box by sorting dimensions (L, W, H) for both the product and the box to account for 3D rotation/orientation. Out of all valid candidate boxes, return the one with the lowest cost. If no box fits, return None. Include clear docstrings and typing annotations."

Response: Here is the implementation for `box_app/services.py` that implements the box selection algorithm based on weight capacity, cumulative volume, 3D item dimension alignment (accounting for rotation), and cost optimization.

`box_app/services.py`

```

from typing import Optional, List, Tuple
from decimal import Decimal
from box_app.models import Order, Box

def _can_product_fit_in_box(
    product_dims: Tuple[Decimal, Decimal, Decimal],

```

```

    box_dims: Tuple[Decimal, Decimal, Decimal]
) -> bool:
    """
    Checks whether a product can physically fit inside a box by comparing
    their sorted 3D dimensions (Length, Width, Height).

    Sorting both sets of dimensions accounts for 3D rotation, ensuring
    the smallest dimension of the product is compared against the smallest
    dimension of the box, and so on.
    """
    # Sort dimensions in ascending order: (min_dim, mid_dim, max_dim)
    sorted_product = sorted(product_dims)
    sorted_box = sorted(box_dims)

    return (
        sorted_product[0] <= sorted_box[0] and # Smallest vs Smallest
        sorted_product[1] <= sorted_box[1] and # Middle vs Middle
        sorted_product[2] <= sorted_box[2]     # Largest vs Largest
    )

def recommend_box_for_order(order: Order) -> Optional[Box]:
    """
    Recommends the lowest-cost box that can safely contain all items in an order.

    Algorithm Constraints:
    1. Weight Filter: Total order weight <= Box max_weight.
    2. Volume Filter: Total order volume <= Box internal volume.
    3. Individual Dimension Filter: Every single product's bounding dimensions
       must fit inside the box's internal dimensions (accounting for 3D rotation).
    4. Cost Optimization: Selects the box with the lowest cost among candidate boxes.

    :param order: The Order model instance containing items to pack.
    :return: The optimal Box instance, or None if no suitable box exists.
    """
    # Pre-fetch order items and associated products to avoid N+1 DB queries
    order_items = order.items.select_related('product').all()

    if not order_items:
        return None

    # Calculate aggregate order metrics
    total_order_weight = order.total_weight
    total_order_volume = order.total_volume

    # Step 1 & 2: Filter candidate boxes by overall weight capacity and total volume
    candidate_boxes = Box.objects.filter(
        max_weight__gte=total_order_weight
    )

    valid_boxes: List[Box] = []

    for box in candidate_boxes:
        # Check overall volume constraint
        if box.volume < total_order_volume:
            continue

        box_dims = (box.length, box.width, box.height)
        all_items_fit = True

        # Step 3: Check individual item dimensional fit (accounting for 3D orientation)
        for item in order_items:
            product = item.product
            product_dims = (product.length, product.width, product.height)

```

```

        if not _can_product_fit_in_box(product_dims, box_dims):
            all_items_fit = False
            break # Fail fast if any product exceeds box dimensions

    if all_items_fit:
        valid_boxes.append(box)

if not valid_boxes:
    return None

# Step 4: Return the candidate box with the lowest cost.
# In case of a tie in cost, pick the one with smaller volume to keep packaging snug.
return min(valid_boxes, key=lambda b: (b.cost, b.volume))

```

Key Algorithm Highlights

- 3D Rotation Handling:** The `_can_product_fit_in_box` helper sorts dimensions for both the product and the box `[dim1, dim2, dim3]`. This allows a product measuring `(10 x 50 x 20)` to fit into a box measuring `(25 x 15 x 55)` without hardcoding specific axis rotations.
- Early Exit Optimization:** The loop uses `break` as soon as a single product fails to fit in a candidate box, avoiding redundant dimension checks.
- Deterministic Tie-Breaking:** If two valid candidate boxes share the exact same lowest cost, `min(..., key=lambda b: (b.cost, b.volume))` picks the smaller volume box to minimize unnecessary void-fill packaging.

*User prompt: box_app/serializers.py using Django REST Framework (rest_framework.serializers). Include serializers for: ProductSerializer (all fields) BoxSerializer (all fields + volume property) OrderItemSerializer (include nested product details or product ID and quantity) OrderSerializer (include nested items, total_weight, and total_volume) A standalone BoxRecommendationRequestSerializer that accepts an order_id (IntegerField or UUID) OR a raw list of product items with quantities to request a recommendation directly.***

Response: Here is the complete implementation for box_app/serializers.py using Django REST Framework.

box_app/serializers.py

```

from rest_framework import serializers
from box_app.models import Product, Box, Order, OrderItem

class ProductSerializer(serializers.ModelSerializer):
    """
    Serializer for the Product model exposing all dimensional attributes.
    """
    volume = serializers.DecimalField(
        max_digits=12,
        decimal_places=2,
        read_only=True
    )

    class Meta:
        model = Product
        fields = [
            'id',
            'name',
            'length',
            'width',
            'height',
            'weight',
            'volume'
        ]

```



```
class BoxSerializer(serializers.ModelSerializer):
    """
    Serializer for the Box model exposing capacity metrics and calculated volume.
    """
    volume = serializers.DecimalField(
        max_digits=12,
        decimal_places=2,
        read_only=True
    )

    class Meta:
        model = Box
        fields = [
            'id',
            'name',
            'length',
            'width',
            'height',
            'max_weight',
            'cost',
            'volume'
        ]

class OrderItemSerializer(serializers.ModelSerializer):
    """
    Serializer for OrderItem, supporting write operations via product ID
    and detailed nested product representations on read operations.
    """
    product = ProductSerializer(read_only=True)
    product_id = serializers.PrimaryKeyRelatedField(
        queryset=Product.objects.all(),
        source='product',
        write_only=True
    )

    class Meta:
        model = OrderItem
        fields = ['id', 'product', 'product_id', 'quantity']

class OrderSerializer(serializers.ModelSerializer):
    """
    Serializer for Order including nested items and calculated total metrics.
    """
    items = OrderItemSerializer(many=True, read_only=True)
    total_weight = serializers.DecimalField(
        max_digits=12,
        decimal_places=2,
        read_only=True
    )
    total_volume = serializers.DecimalField(
        max_digits=12,
        decimal_places=2,
        read_only=True
    )

    class Meta:
        model = Order
        fields = [
            'id',
            'created_at',
            'items',
            'total_weight',
            'total_volume'
        ]
```

]

```
class RawOrderItemInputSerializer(serializers.Serializer):
    """
    Helper serializer for validating raw product item inputs
    when requesting box recommendations without creating an Order object first.
    """
    product_id = serializers.IntegerField(min_value=1)
    quantity = serializers.IntegerField(min_value=1, default=1)

class BoxRecommendationRequestSerializer(serializers.Serializer):
    """
    Flexible request serializer accepting either an existing order_id
    OR a list of raw product items with quantities.
    """
    order_id = serializers.IntegerField(
        required=False,
        allow_null=True,
        min_value=1
    )
    items = RawOrderItemInputSerializer(
        many=True,
        required=False
    )

    def validate(self, attrs):
        order_id = attrs.get('order_id')
        items = attrs.get('items')

        # Require exactly one approach: order_id OR raw items
        if not order_id and not items:
            raise serializers.ValidationError(
                "Must provide either 'order_id' or 'items'."
            )
        if order_id and items:
            raise serializers.ValidationError(
                "Provide either 'order_id' or 'items', not both."
            )

        # Validate that the order exists if order_id is given
        if order_id:
            if not Order.objects.filter(id=order_id).exists():
                raise serializers.ValidationError({
                    'order_id': f"Order with ID {order_id} does not exist."
                })

        # Validate product_ids if raw items are given
        if items:
            product_ids = [item['product_id'] for item in items]
            existing_products = set(
                Product.objects.filter(id__in=product_ids).values_list('id', flat=True)
            )
            missing_ids = set(product_ids) - existing_products
            if missing_ids:
                raise serializers.ValidationError({
                    'items': f"Products with IDs {list(missing_ids)} do not exist."
                })

        return attrs
```

Highlights & Validation

1. **Dynamic Payload Flexibility:** BoxRecommendationRequestSerializer allows API consumers to send either {

```
"order_id": 12 } or { "items": [{"product_id": 1, "quantity": 2}] }.
```

2. **Mutual Exclusivity Validation:** Overridden `validate()` ensures clients cannot supply both or neither parameter.

3. **Foreign Key Integrity Verification:** Checks directly whether specified `order_id` or `product_id` instances exist in the database before processing business logic.

*User prompt: box_app/views.py file using Django REST Framework. Create ModelViewSets or APIViews for managing Product, Box, and Order objects. Create an @api_view(['POST']) endpoint named recommend_box_view or an APIView class /api/recommend-box/. The view should accept an order_id in the request body, retrieve the Order instance, invoke recommend_box_for_order(order) from services.py, and return the recommended box details as JSON with HTTP status 200. If no box is found, return {"message": "No suitable box found for this order."} with HTTP 404 or 200 depending on standard API design. Handle invalid order IDs cleanly with HTTP 400/404.***

Response: Here is the complete implementation for box_app/views.py using Django REST Framework.

box_app/views.py

```
from rest_framework import status, viewsets
from rest_framework.decorators import api_view
from rest_framework.request import Request
from rest_framework.response import Response

from box_app.models import Product, Box, Order
from box_app.serializers import (
    ProductSerializer,
    BoxSerializer,
    OrderSerializer,
    BoxRecommendationRequestSerializer,
)
from box_app.services import recommend_box_for_order

class ProductViewSet(viewsets.ModelViewSet):
    """
    CRUD API endpoint for managing Products.
    """
    queryset = Product.objects.all()
    serializer_class = ProductSerializer

class BoxViewSet(viewsets.ModelViewSet):
    """
    CRUD API endpoint for managing Boxes.
    """
    queryset = Box.objects.all()
    serializer_class = BoxSerializer

class OrderViewSet(viewsets.ModelViewSet):
    """
    CRUD API endpoint for managing Orders.
    """
    queryset = Order.objects.prefetch_related('items__product').all()
    serializer_class = OrderSerializer

@api_view(['POST'])
def recommend_box_view(request: Request) -> Response:
    """
    Recommends the cheapest suitable box for a given order or raw list of items.

    Request Body Examples:
```

```

1. By Order ID:
    { "order_id": 1 }

2. By Raw Items:
    { "items": [{ "product_id": 1, "quantity": 2 }] }
"""

serializer = BoxRecommendationRequestSerializer(data=request.data)
if not serializer.is_valid():
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

data = serializer.validated_data
order_id = data.get('order_id')
items = data.get('items')

# Handle transient/ephemeral order calculation without modifying the DB
if items:
    # Create an unsaved Order instance to leverage model helper methods
    order = Order()
    order.save = lambda *args, **kwargs: None # Prevent saving

    # Instantiate in-memory items
    product_ids = [item['product_id'] for item in items]
    products_map = {p.id: p for p in Product.objects.filter(id__in=product_ids)}

    # Dynamically attach items mock query set / list
    mock_items = []
    for item in items:
        product = products_map[item['product_id']]
        mock_item = type('MockOrderItem', (), {
            'product': product,
            'quantity': item['quantity']
        })
        mock_items.append(mock_item)

    # Override properties for in-memory recommendation calculation
    order.total_weight = sum((i.product.weight * i.quantity for i in mock_items), Decimal('0.00'))
    order.total_volume = sum((i.product.volume * i.quantity for i in mock_items), Decimal('0.00'))
    order.items = type('MockManager', (), {
        'select_related': lambda *args, **kwargs: type('MockQS', (), {'all': lambda: mock_items})()
    })()
else:
    # Fetch existing order from DB
    try:
        order = Order.objects.get(id=order_id)
    except Order.DoesNotExist:
        return Response(
            {"error": f"Order with ID {order_id} not found."},
            status=status.HTTP_404_NOT_FOUND
        )

# Invoke algorithm from services
recommended_box = recommend_box_for_order(order)

if not recommended_box:
    return Response(
        {"message": "No suitable box found for this order."},
        status=status.HTTP_200_OK
    )

box_serializer = BoxSerializer(recommended_box)
return Response(
    {
        "recommended_box": box_serializer.data,
        "message": "Box recommendation found successfully."
    },

```

```

        status=status.HTTP_200_OK
    )

```

Step-by-Step URL Routing (box_app/urls.py)

To route these views, create box_app/urls.py:

```

from django.urls import path, include
from rest_framework.routers import DefaultRouter
from box_app.views import ProductViewSet, BoxViewSet, OrderViewSet, recommend_box_view

router = DefaultRouter()
router.register(r'products', ProductViewSet, basename='product')
router.register(r'boxes', BoxViewSet, basename='box')
router.register(r'orders', OrderViewSet, basename='order')

urlpatterns = [
    path('', include(router.urls)),
    path('recommend-box/', recommend_box_view, name='recommend-box'),
]

```

Then hook box_app/urls.py into your main box_selection_system/urls.py:

```

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('box_app.urls')),
]

```

User prompt: how shall i export the chat transcript ?

Response: Depending on which AI tool you are using for your development, here is how you can export your chat transcripts so you can save them directly into your repository (e.g., inside a docs/chat_transcripts/ folder) as required by the assignment guidelines.

1. ChatGPT (OpenAI)

ChatGPT offers a built-in data export feature or a direct link-sharing option:

- **Option A (Settings Export):** Go to your **Profile / Settings -> Data Controls -> Export Data**. OpenAI will email you a zip file containing your complete chat history in JSON format. You can locate your specific session file and add it to your repo.
- **Option B (Manual Markdown/Text Save):** For a single specific conversation thread, you can use browser extensions (like "ChatGPT to Markdown" or "Export ChatGPT to PDF/PNG") or simply copy the core prompt-response history into a .md (Markdown) or .txt file.

2. Claude (Anthropic)

- **Manual Export:** Claude currently doesn't have a single-click "Export Transcript" button for individual chats. The standard practice is to copy the conversation or use browser-based extensions to save the thread as a Markdown (.md) or text file. Alternatively, you can highlight the interaction, paste it into a file named `claude_transcript_session_1.md`, and place it in your repository.

3. Cursor / GitHub Copilot (IDE-based Assistants)

Since Cursor and Copilot live directly inside your code editor:

- **Cursor:** Cursor stores chat histories locally. You can access your past Composer or Chat sessions via the sidebar history, copy the relevant logs, and paste them into a markdown file (e.g., `cursor_chat_history.md`).
- **GitHub Copilot:** Copilot doesn't maintain a long-form historical chat transcript file by default, but if you used the Copilot Chat panel, you can copy the relevant query/response blocks that directly influenced your code and save them into an

ide_chat_log.txt file.

💡 Recommendation for Your Repo

Create a dedicated directory in your project root to keep things clean and organized for the reviewers:

```
box_selection_system/  
├── docs/  
│   └── chat_transcripts/  
│       ├── chatgpt_session_models.json (or .md)  
│       └── claude_session_algorithm.md
```

User prompt: in gemini ?

Response: Because Google Gemini does not have a single-click "Download full conversation transcript" button for individual threads, you can use one of these practical workarounds to save your chat log for your repository:

Method 1: The "Export to Google Docs" Feature (Easiest Built-in Option)

1. Scroll to the bottom of individual responses or hover over them.
2. Click the **Share & export** icon (or the three vertical dots under a response).
3. Select **Export to Docs**. This instantly creates a clean, formatted Google Document in your Google Drive.
4. From that Google Doc, you can go to **File > Download > Markdown (.md)** or **Plain Text (.txt)** and drop that file right into your repository's docs/chat_transcripts/ folder.

Method 2: Manual Markdown or Text File

Since the assignment explicitly mandates a raw text or markdown transcript that looks unmodified by AI, manually compiling the thread is fully acceptable and safe:

1. Copy the relevant back-and-forth prompts and responses from this chat session.
2. Paste them into a text file or markdown file (e.g., gemini_chat_transcript.md).
3. Save it directly in your repo structure under docs/chat_transcripts/gemini_chat_transcript.md.

Method 3: Google Takeout (For Full Bulk Account History)

If you want an official automated archive of all your past prompts across Gemini:

1. Go to takeout.google.com.
 2. Click **Deselect all**, scroll down, and check **My Activity** (or **Gemini / Gemini Apps**).
 3. Complete the export to receive a download link via email containing your raw chat logs in HTML/JSON format.
-